Lab 01 — Questions

Answer without re-reading the theory. One to five sentences are enough; what counts is the reasoning, not the vocabulary.

Question 1 [Understanding]

A colleague states: "A container is a small virtual machine with a minimal Linux inside." Say what is wrong in that sentence, and explain why this confusion has concrete consequences on start-up time and disk usage.

Question 2 [Analysis]

The `postgres:16-alpine` image weighs about 250 MB and contains a complete Linux tree (`/bin`, `/etc`, `/usr`...). Yet we say "there is no OS in a container". Both statements are true: explain what is really inside that image and what, in an operating system, is **absent** from it.

Question 3 [Analysis]

You start two containers from the same `nginx:alpine` image. On the host, `ps aux | grep nginx` shows two sets of processes. Yet from inside the first container, `ps` only shows its own processes. Which kernel mechanism is responsible, and why is it **not** security in the strict sense?

Question 4 [Diagnosis]

A colleague who uses Docker shows you this:

```
Client: Docker Engine - Community
 Version:  29.7.2
Cannot connect to the Docker daemon at unix:///var/run/docker.sock.
Is the docker daemon running?
```

Explain what happened on their machine (two likely causes), why changing the command is pointless — and why this message can **not** happen to you with your rootless Podman under WSL.

Question 5 [Understanding]

"An image does not run." Justify that sentence, then explain what the engine concretely adds to the image when you type `podman run`.

Question 6 [Analysis]

You start a PostgreSQL container, create a database and tables inside it, then `podman rm` that container. You start a new container from the **same image**. Is your data there? Justify using the image / writable layer structure — and say whether the image was modified by your work.

Question 7 [Analysis]

You start ten containers from the `eclipse-temurin:21-jre-alpine` image (about 210 MB). Approximately how much additional disk space does that consume? Explain the mechanism that makes this answer possible.

Question 8 [Diagnosis]

Inside a rootless Podman container, `id` prints `uid=0(root)`. On the WSL host, `podman top <container> user,huser` prints `root` in the USER column and `1000` in the HUSER column. Explain

what this double identity means, which namespace produces it, and what that "root" can really do if it tries to write to the host's `/etc/shadow` through a mount.

Question 9 [Understanding]

Your company forbids adding users to the `docker` group on production servers, and requires going through `sudo` with an audit trail. What is the security reasoning behind that rule, and how does rootless Podman make the question moot?

Question 10 [Analysis]

Translate the following commands into their long form (`podman <object> <action>`), and say for each which **type of object** it acts on:

```
podman ps -a
podman images
podman rmi nginx:alpine
podman rm web
```

Why don't `podman ps` and `podman images` follow the same naming logic?

Question 11 [Diagnosis]

From your Ubuntu terminal under WSL, `podman run --rm alpine uname -r` prints `6.6.87.2-microsoft-standard-WSL2`. A colleague on a native Ubuntu server gets `6.8.0-45-generic` with the same command. Explain where each value comes from, and what that implies for the claim "containers are lightweight" on a Windows workstation.

Question 12 [Analysis]

A badly written containerised application goes into an infinite loop and consumes all available RAM. Does the `pid` namespace stop it from harming the other containers? What is the right mechanism to use, and what happens if nobody configured it?

Question 13 [Understanding]

In the workplace, the Spring Boot back-end image built by CI is promoted from integration to staging to production **without being rebuilt**, and the staging servers run Docker while production runs Podman. Which properties make this practice possible, and what risk do you take if you rebuild the image at each stage from the same source code?

Question 14 [Analysis]

A developer adds `alias docker=podman` to their `.bashrc` and claims "everything written for Docker will work". Give two examples where that is true without reservation, and two situations where Podman's different architecture (no daemon, rootless) concretely changes the observed behaviour.